

Advanced DevOps

WELCOME TO CLASS 10!



Goals of the day



Other DevOps tool options



CI/CD & Serverless



GitOps, MLOps, AIOps



Photo by [David Becker](#) on [Unsplash](#)

©Maja Bjeletic@unsplash

Agenda

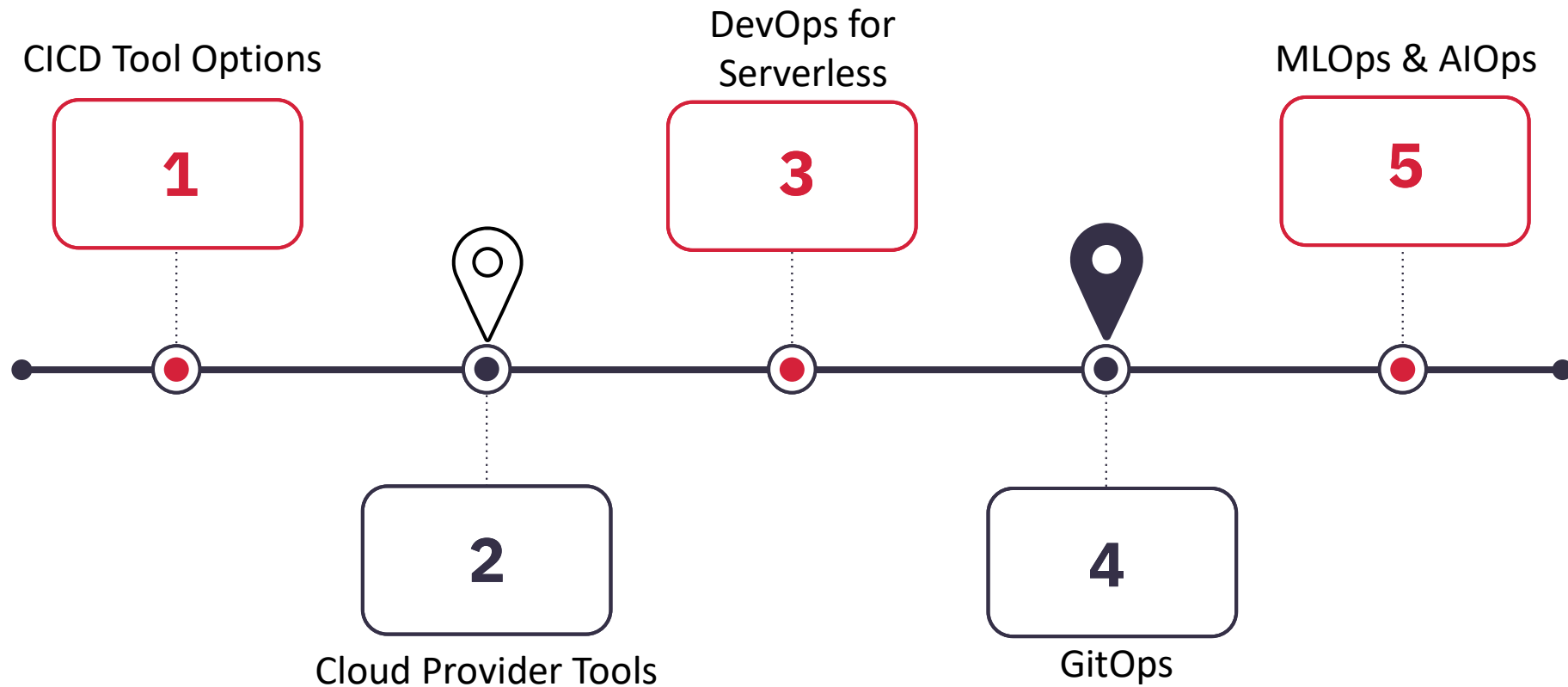
- CICD tooling options
- **Break**
- Cloud provider DevOps tooling
- DevOps for serverless applications
- GitOps
- MLOps & AIOps

Photo by David Becker on Unsplash

Review

- At this point we have covered the detailed technical material for this module. You have reviewed around 20 code repos designed for learning git, Terraform, Ansible & Jenkins.
- Beyond this core toolset, there are many other options to achieve the same things.
- From what you have seen to date, are there any pros and cons for these tools that you'd like to call out?
- Have you used any other equivalent tools that you can compare against what we have learned here?
- Any other comments as we begin the last third of this module?

Journey Map



CICD Tooling Options

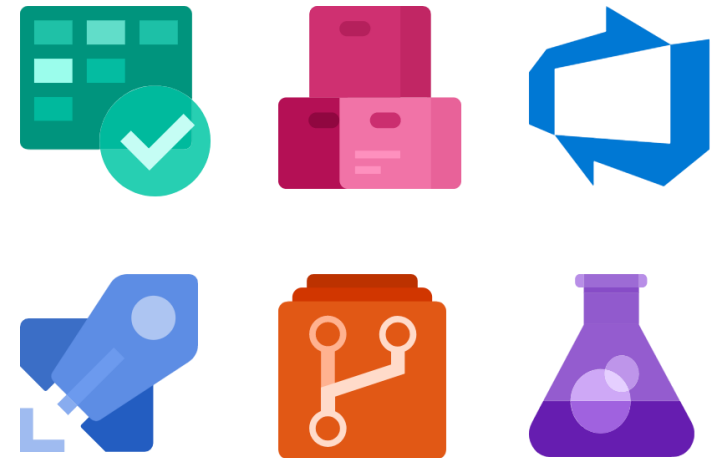
- Jenkins is the market leader amongst CICD tools, but there are other options, which are increasing market share year on year.
- In this section we will review GitHub Actions and Azure DevOps, which are two popular alternatives and then ask you to compare them with the market leader.
- The principles and knowledge you have acquired on how to create great CICD pipelines is transferrable between these tools.



<https://www.jenkins.io/artwork/>

Azure DevOps

- Azure DevOps is a collection of products, which includes the following:
 - Boards – agile tooling for tracking/planning work
 - Repos – git repository for version control
 - Pipelines – supports continuous integration and delivery of applications. Jenkins equivalent product
 - Test plans – supports manual or automated testing
 - Artifacts – package storage and sharing across Maven, npm, NuGet repository types
 - Azure DevOps is free for up to 5 users per organization.



Azure Pipelines

- Focusing on Pipelines, which is available for any git SCM, it enables building, scanning, testing, archiving and deploying your application. The key elements are:
 - Pipeline – configured as yml code that is checked directly into the source code repo
 - Stage – contains one or more jobs, a default is used if no stage is provided
 - Jobs – contain one or more tasks. Jobs are run on agents and can be sequenced or run in parallel
 - Tasks – the execution part of the pipeline, with inputs for each task provided in the yml code.



Azure Pipelines

<https://devblogs.microsoft.com/premier-developer/azure-devops-pipelines-tasks-jobs-stages-and-more/>

Azure Pipelines - Execution

- This sample code shows a pipeline to run unit tests as part of a Spring Boot build
 - Trigger – commit to master will execute this pipeline
 - Pool – the agent on which this pipeline will execute
 - Steps/task – running the Maven task with inputs provided including specifics around test output
- This pipeline will publish test results within Azure Pipelines as part of the job execution output.

```
1  # Maven
2  # Build your Java project and run tests with Apache Maven.
3  # Add steps that analyze code, save build artifacts, deploy, and more:
4  # https://docs.microsoft.com/azure/devops/pipelines/languages/java
5
6  trigger:
7    - master
8
9  pool:
10     vmImage: 'ubuntu-latest'
11
12  steps:
13    - task: Maven@3
14      inputs:
15        mavenPomFile: 'pom.xml'
16        mavenOptions: '-Xmx3072m'
17        javaHomeOption: 'JDKVersion'
18        jdkVersionOption: '1.11'
19        jdkArchitectureOption: 'x64'
20        publishJUnitResults: true
21        testResultsFiles: '**/surefire-reports/TEST-*.xml'
22        goals: 'package'
```

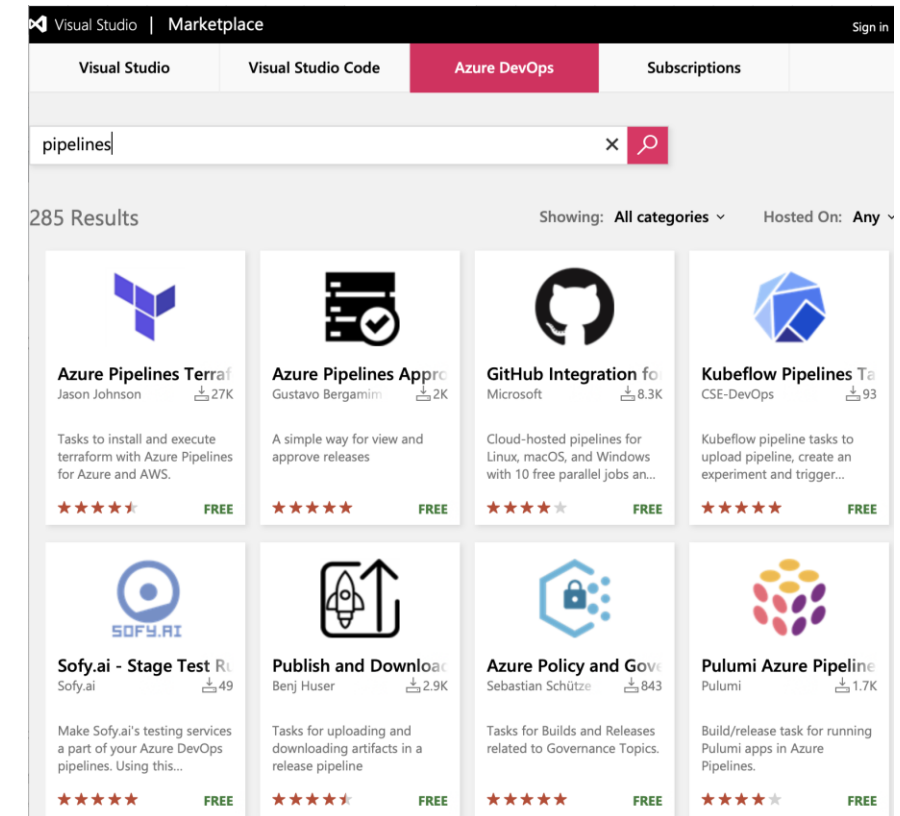
<https://medium.com/@TimvanBaarsen/build-your-spring-boot-project-using-azure-pipelines-in-azure-devops-3305977991d>

Azure Pipelines – Custom Tasks

- Within the Azure DevOps ecosystems, there is the option to create custom extensions, a subset of which is the ability to create custom tasks. *
- These custom tasks are developed in nodejs, packaged then published to the Visual Studio Marketplace. **
- Create a build and release pipeline to manage testing and publication of your custom task to the marketplace.
- Once published you can then install the extension within your organization's Azure DevOps instance and reference it from a pipeline yaml file.

* <https://learn.microsoft.com/en-us/azure/devops/extend/overview?view=azure-devops>

** <https://learn.microsoft.com/en-us/azure/devops/extend/develop/add-build-task>

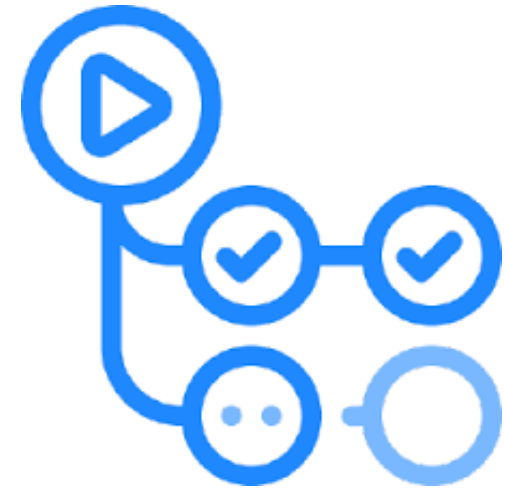


<https://marketplace.visualstudio.com/search?term=pipelines&target=AzureDevOps&category=All%20categories&sortBy=Relevance>

GitHub Actions

This is a very valuable add on feature for those hosting their code in GitHub. Limited to code in GitHub repos.

- The important CI/CD outcomes such as building, scanning, testing, archiving and deploying your application can all be achieved.
- Pipelines are configured as yml code that is checked directly into the source code repo in a `.github/workflows` folder.
- Workflows contain one or more jobs, which are triggered by events, such as code commit or branch merge.
- Jobs contain one or more steps, which are configured as shell script execution or configuration for an Action.



GitHub Actions

GitHub Actions - Execution

- GitHub Actions is a SaaS solution, where your workflows are executed on runners that are specified at the job level.
- It is free for public repos, with limited free usage for private repos.
- The code screenshot is for a continuous integration pipeline for a Spring Boot application.
- It is triggered by 2 events, either pull request or push to **main** branch.
- It contains one build job which has several steps.

```
name: Build and Test Java Spring Boot Application

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:

    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven

      - name: Build with Maven
        run: mvn clean verify

      - name: Test Report
        uses: actions/upload-artifact@v3
        if: success() || failure()
        with:
          name: SpringBoot Report           # Name of the folder
          path: target/site/serenity/       # Path to test results
```

<https://qaautomation.expert/2023/06/26/how-to-run-springboot-tests-with-github-actions/>

GitHub Actions – Execution (con't)

- Runners are machines that are either provided by GitHub or you can also configure your own custom machines. Here the runner is **ubuntu-latest**.
- The **uses** statements reference publicly available actions for almost anything you would need to do.
- The **run** command allows execution of shell statements.
- Here the build environment is setup on the runner, the code is packaged and tested using the **mvn clean verify** command.
- The test results are then published on GitHub using the **upload-artifact** action.

```
name: Build and Test Java Spring Boot Application

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:

    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven

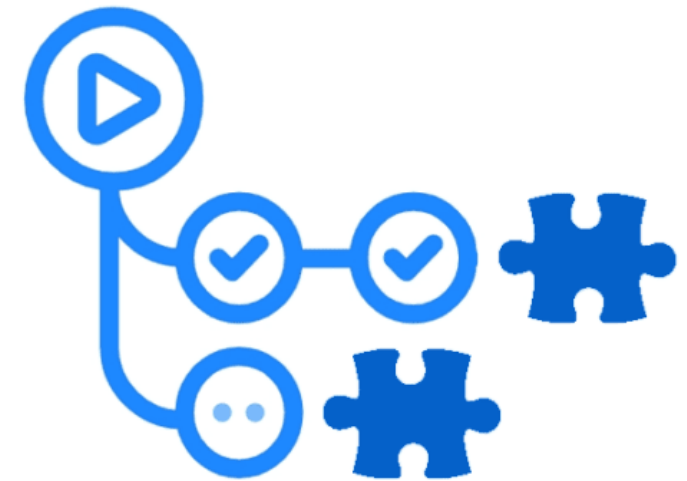
      - name: Build with Maven
        run: mvn clean verify

      - name: Test Report
        uses: actions/upload-artifact@v3
        if: success() || failure()
        with:
          name: SpringBoot Report           # Name of the folder
          path: target/site/serenity/       # Path to test results
```

<https://qaautomation.expert/2023/06/26/how-to-run-springboot-tests-with-github-actions/>

GitHub Actions – Custom Actions

- In addition to the publicly available actions, there is also a marketplace of actions developed by others, plus the option to create your own actions.
- Options for developing custom actions include JavaScript, Docker containers, or a composite action, which is a combination of existing actions.
- JavaScript functions can be added directly to your code under a .github/actions folder
- Docker containers allow you to create a container using any image you wish, which offers a lot of flexibility, including which language you use.



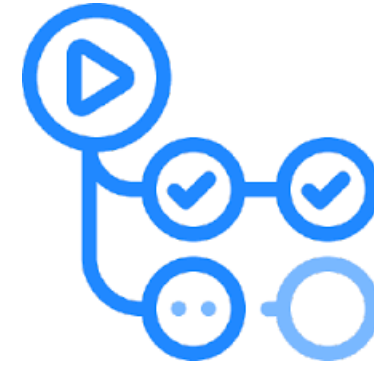
**GitHub Custom
Actions**

<https://dev.to/oneadvanced/github-reusable-workflows-and-custom-actions-3cbk>

Comparison

How do each of these tools compare across the following data points?

- Total cost of ownership (licenses, maintenance, etc.)
- Ease of use of agents
- Custom code creation
- Ease of integration across different SCMs
- Any other thoughts?



GitHub Actions



Jenkins



Azure Pipelines

Cloud Provider DevOps tooling

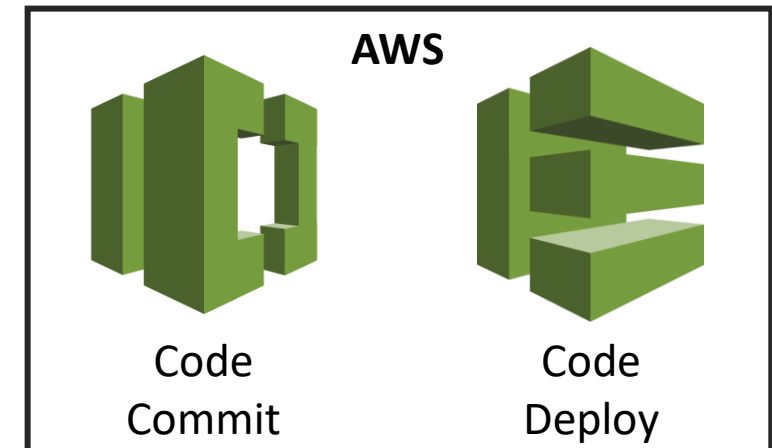
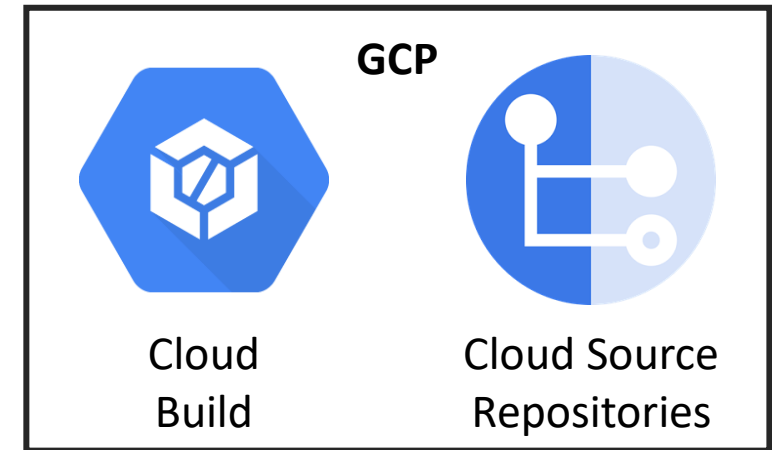
AWS & GCP tooling

In addition to the leading CI/CD products, a review of product offerings from Amazon Web Services and Google Cloud Platform will give you a broader perspective on tooling.

Both platforms have git repos and CI/CD solutions.

For most organizations, their first use of cloud will be with a single vendor, which makes using available tooling seem like a good idea.

Most organizations will evolve to a multi-cloud setup at some point, so choosing an agnostic solution for DevOps tooling may be wise in the long term.



GCP tooling

Code Build can import code from a variety of SCMs or cloud storage.

- fetch dependencies, run unit tests, static analyses, and integration tests, and create artifacts with main build tools
- option to use community build steps or wrote custom code

Cloud Source Repositories is a git based SCM, replacing Google Cloud

- Allows mirroring with bidirectional synchs across GitHub/Bitbucket
- Includes integrations with other GCP tools to speed build, test and deploy on the platform.
- Unlimited private repos.

**A vendor agnostic CI/CD tool may be more strategic for longer term.
git solution is open in design, so worth considering.**



Cloud
Build



Cloud Source
Repositories

AWS tooling

Code Commit is a git based SCM run on managed AWS infrastructure.

- Favored for it's higher level of security than alternatives as it uses AWS's IAM Roles and users.
- Pay as you go service.
- Good integration with other AWS products.

Code Deploy is a deployment service for AWS and On Prem infra

- Connects to multiple git based SCMs
- Integration with AWS Lambda functions
- Supports multiple deployment patterns e.g., blue-green.

Similarly, a vendor agnostic CI/CD tool may be more strategic for longer term. git solution is open in design, so worth considering.



Code
Commit



Code
Deploy

DevOps for Serverless Applications

DevOps for Serverless

With serverless compute, much of the admin overhead is managed by the platform and so the infrastructure component is greatly simplified.

DevOps practices offer many benefits for this type of application, particularly with the ease of deployment being amplified by great CICD pipelines.

- With such ease of use, appropriate governance must be applied to ensure that this approach is managed effectively e.g., old applications removed.
- Good starting point for a platform engineering team approach, to make serverless deployments available to engineers, but with guardrails in place.
- Available storage is limited in lambda, which is a problem if the application requires large libraries included in deployment.



<https://akfpartners.com/growth-blog/4-landmines-when-using-serverless-architecture>

Next, GitOps

GitOps (1 of 3)

"GitOps is the best thing since configuration as code. Git changed how we collaborate, but the declarative configuration is the key to dealing with infrastructure at scale and sets the stage for the next generation of management tools." Kelsey Hightower

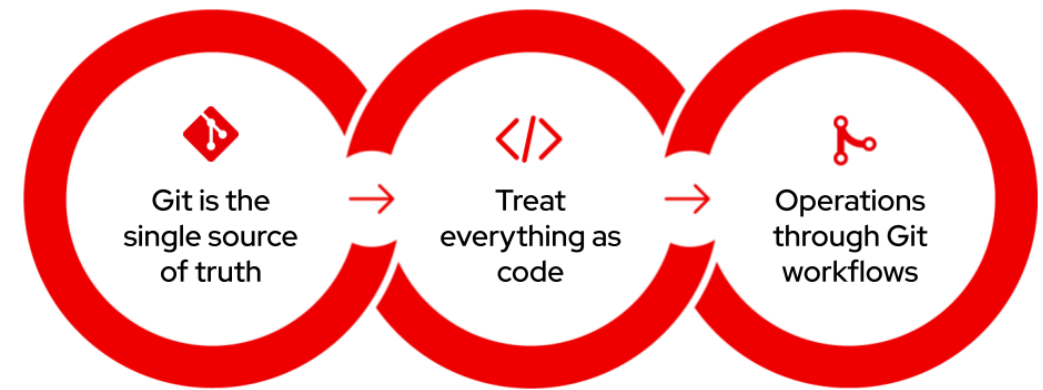


Kelsey Hightower, former Distinguished Engineer at Google Cloud

GitOps (2 of 3)

GitOps is an operating model for building cloud infrastructure that unifies deployment, monitoring and management.

- Git becomes the single source of truth for the deployed infrastructure.
- Changes to git are verified through CI pipeline and automatically applied to the infrastructure through CD pipeline
- Divergence of infrastructure from declared state in git is automatically remediated
- Change management workflow is overlayed on git workflow, with an audit trail that clearly shows what happened.



<https://cloud.redhat.com/blog/openshift-pipelines-and-openshift-gitops-are-now-generally-available>

GitOps (3 of 3)

GitOps Benefits *

- Increased Productivity
- Enhanced Developer Experience
- Improved Stability
- Higher Reliability
- Consistency and Standardization
- Stronger Security Guarantees

GitOps is often associated with Kubernetes, as some of the prominent GitOps tools such as Flux and Argo are deployed as containers.



<https://hackernoon.com/why-gitops-is-so-exciting-ca2a3a0y>

* [https://www.ansible.com/hubfs/Webinar%20PDF%20slides/Ops%20by%20Pull%20Request%20An%20Ansible%20GitOps%20Story%20\(Ansible%20Webinar%20August%202020\).pdf](https://www.ansible.com/hubfs/Webinar%20PDF%20slides/Ops%20by%20Pull%20Request%20An%20Ansible%20GitOps%20Story%20(Ansible%20Webinar%20August%202020).pdf)

GitOps Workflow *

Infrastructure as Code

- Provisioning and configuration management through IaC is the foundation for your GitOps practice.

Merge Requests

- With the main branch declared as state for infrastructure, branch updates impact infrastructure. Merge requests provide control.

CI/CD Pipelines

- Main branch updates trigger CI pipelines, which if successful results in rollout of changes through CD pipelines.

Monitoring

- Knowing that the infrastructure is out of alignment with the declared state in git triggers auto-healing to that state.



<https://hackernoon.com/why-gitops-is-so-exciting-ca2a3a0y>

* <https://about.gitlab.com/topics/gitops/gitops-workflow/>

GitOps Benefits *

- Increased Productivity
- Enhanced Developer Experience
- Improved Stability
- Higher Reliability
- Consistency and Standardization
- Stronger Security Guarantees



<https://www.harness.io/blog/gitops-principles>

GitOps is often associated with Kubernetes, as some of the prominent GitOps tools such as Flux and Argo are deployed as containers. We will look at this further in module 6.

MLOps & AIOps

MLOps & AIOps (con't)

Machine Learning Ops and Artificial Intelligence Ops sound like the same thing, however they are quite different. This section will provide an overview of both.

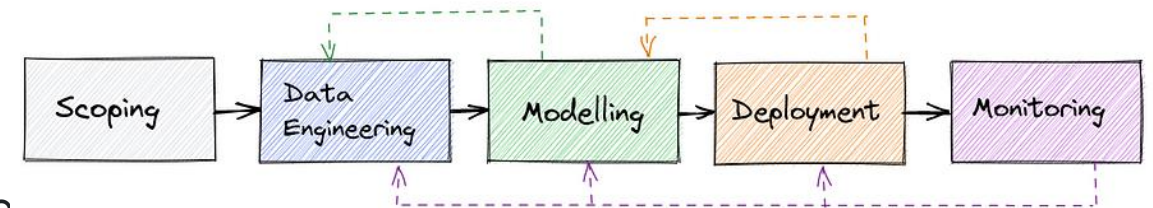


Image by Dall-E

MLOps

Machine learning is a subfield of artificial intelligence that gives computers the ability to learn without explicitly being programmed. *

- Large amounts of data are curated as inputs for the modelling process.
- The output from this process is a machine learning model, which is considered to correlate closely with the behaviours observed in the data on which it was trained.
- That model is subsequently deployed so that it can be used by other applications and then monitored for health and statistical consistency



<https://towardsdatascience.com/a-gentle-introduction-to-mlops-7d64a3e890ff>

These processes are enhanced by applying DevOps, becoming MLOps processes in this context.

MLOps (con't)

As a goal, MLOps seeks to automate the complete Machine Learning workflow, without any manual intervention. *

- **Data pipeline:** regularly sourcing data (daily, weekly, etc.) from a large data store, such as a data warehouse and processing to land it in a suitable format for machine learning.
- **Model pipeline:** training of the model can be automated, as well as validation of generated model and underlying data. Often created as Python code, CICD practices are beneficial.
- **Code pipeline:** models often have supporting libraries built by the Data Science team, which also need CICD pipelines and structured archiving.

Version control of models, training scripts and data sets is part of practice, ensuring traceability of any model at any time.



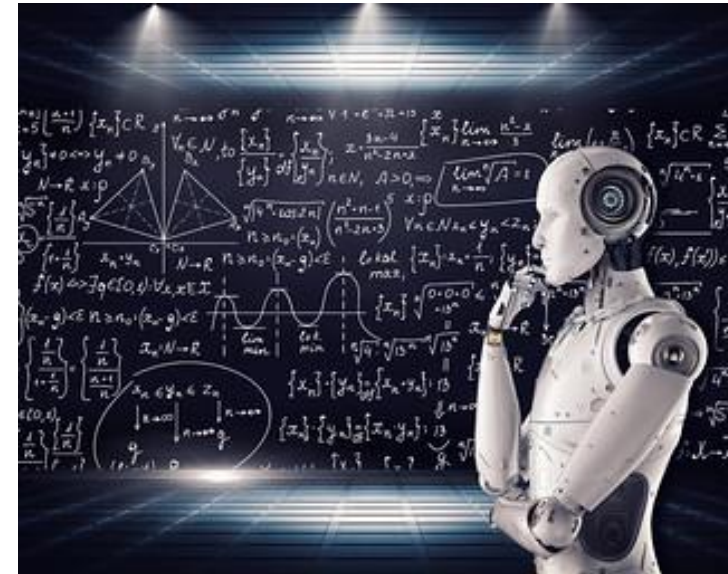
Image by Dall-E

* <https://ml-ops.org/content/mlops-principles>

AIOps

Artificial intelligence for IT Operations (AIOps) is the application of AI practices to traditional IT Ops activities and tasks. *

- IT Operations generate huge volumes of data across application, OS and infrastructure logs such that reviewing these logs manually to generate insight is not feasible.
- AI Ops applies the power of AI to interpreting these log files, working out patterns across all of the available data that indicate issues are likely to occur.
- These insights are tremendously valuable in creating truly self-healing infrastructure that triggers remediation ahead of degradation in performance.



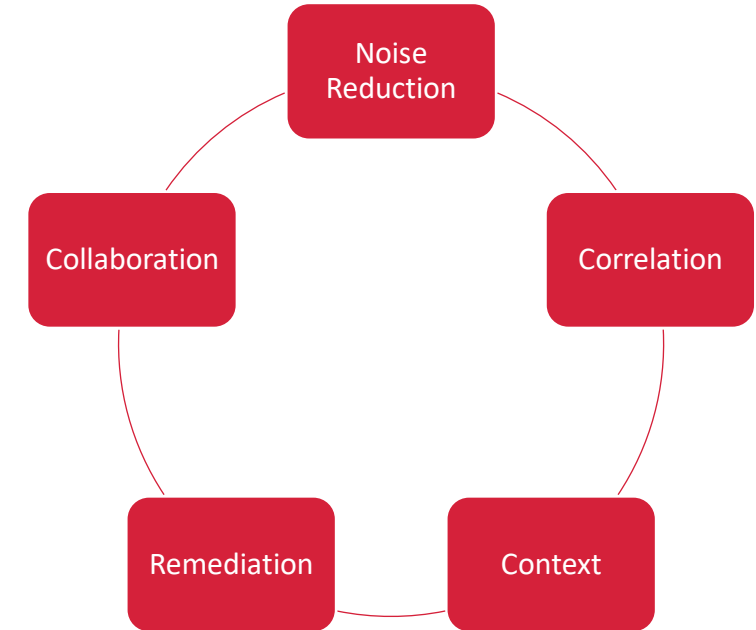
This Photo by Unknown Author is licensed under [CC BY](#)

* <https://www.moogsoft.com/everything-aiops-guide/>

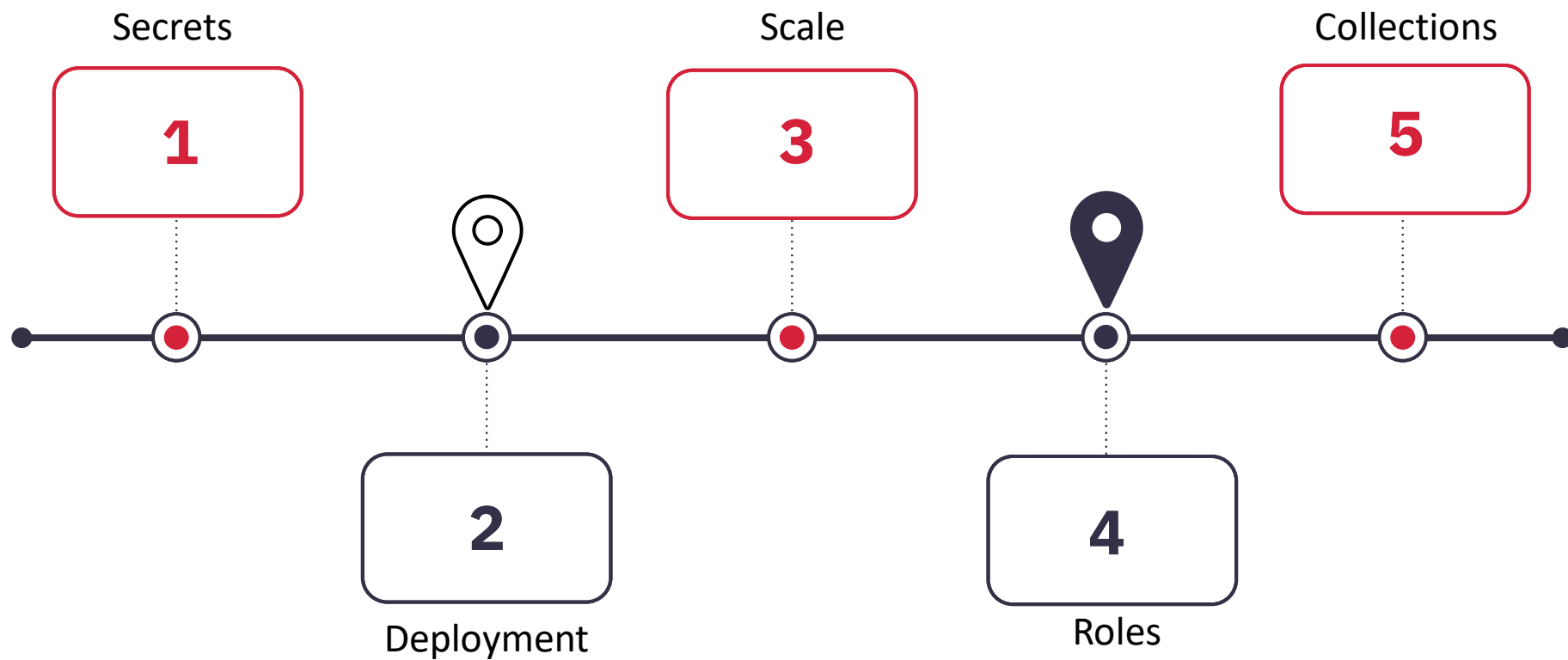
AIOps (con't)

The key capabilities of AIOps are: *

- Noise Reduction: identifies relevant information and filters out the noise
- Correlation: spanning multiple different data sources that may seem to be unrelated
- Context: assessing application, infrastructure user sentiment scores and utilising ML on this data to provide context
- Remediation: for recurring incidents, solutions can be automated, to create true self-healing environments.
- Collaboration: through insight into connections n disparate data sources, AIOps facilitates collaboration amongst different technology owners.



Journey Map



Appendix (for use by Instructor in completing coding). To be removed or modified and added to main deck.

